

The `transform` function you implemented failed on at least one of the examples you were provided.

Your task is to determine what the issue is and then fix the code. The issue could be a bug in the code and/or an issue with your previous understanding of the transformation rule.

You'll need to carefully reason to determine the issue and to determine how to fix the code. Start your response by doing this reasoning in `<fix_reasoning></fix_reasoning>` tags. Then, implement the fixed transformation in code.

I have asked you this same question before. I am showing you many of your attempts to solve this problem that were incorrect in some way. I am including your entire response, including your reasoning that led to the wrong `transform` function.

Here are the examples that have failed:

Your previous incorrect response #0:

Let me analyze the examples to find the pattern of transformations:

1. First Example:

- Has a green (3) diagonal line pattern
- In the output, two cells changed from black (0) to yellow (4)
- These yellow cells are in empty spaces completely surrounded by green cells (have green cells orthogonally adjacent)

2. Second Example:

- Has a scattered pattern of green cells
- One cell changes to yellow (4)
- The changed cell is again in an empty space completely surrounded by green cells (orthogonally)

3. Third Example:

- Has several connected green structures
- Multiple cells change from black (0) to yellow (4)
- Forms a large block of yellow cells in the middle
- Each yellow cell is again surrounded by green cells orthogonally

4. Fourth Example:

- Has a more complex pattern with multiple green structures
- Several cells change from black to yellow

- Again, each yellow cell in the output is surrounded by green cells orthogonally
- Forms two 2x2 yellow blocks and one single yellow cell

5. Fifth Example:

- Has multiple connected green structures
- Many cells change from black to yellow
- Forms several blocks of yellow cells
- Each yellow cell is surrounded by green cells orthogonally

The transformation rule appears to be:

1. Look for empty spaces (black cells, value 0) that are completely surrounded by green cells (value 3) orthogonally (up, down, left, right)
2. Change these surrounded empty spaces to yellow cells (value 4)
3. The rest of the grid remains unchanged

To verify this rule:

- In all examples, every cell that changes from black to yellow is surrounded by green cells
- All cells that are surrounded by green cells orthogonally change to yellow
- No other cells change color
- The pattern holds consistently across all examples
- Green cells and other colored cells remain unchanged

This rule explains all the transformations we see in the examples and appears to be complete with no exceptions.

```
def transform(grid: list[list[int]]) -> list[list[int]]:
    # Convert to 2D numpy array for easier manipulation
    import numpy as np
    grid = np.array(grid)
    rows, cols = grid.shape

    # Create a copy of the grid to store the result
    result = grid.copy()

    # Check each cell in the grid
    for i in range(rows):
        for j in range(cols):
            # Skip if cell is not empty (black)
            if grid[i, j] != 0:
                continue

            # Check if cell is surrounded by green cells
            is_surrounded = True
```

```

# Check up
if i == 0 or grid[i-1, j] != 3:
    is_surrounded = False
# Check down
if i == rows-1 or grid[i+1, j] != 3:
    is_surrounded = False
# Check left
if j == 0 or grid[i, j-1] != 3:
    is_surrounded = False
# Check right
if j == cols-1 or grid[i, j+1] != 3:
    is_surrounded = False

# If surrounded by green cells, change to yellow
if is_surrounded:
    result[i, j] = 4

return result.tolist()

```

The results of using your `transform` function on each example:

Example 0:

Input ASCII representation:

```

0|0|0|0|0|0
0|0|3|0|0|0
0|3|0|3|0|0
0|0|3|0|3|0
0|0|0|3|0|0
0|0|0|0|0|0

```

Expected Output ASCII representation:

```

0|0|0|0|0|0
0|0|3|0|0|0
0|3|4|3|0|0
0|0|3|4|3|0
0|0|0|3|0|0
0|0|0|0|0|0

```

Your `transform` function was correct on this example! Think about why it worked on this and not others.

Example 1:

Input ASCII representation:

```
0|0|0|0|0|0|0|0|0|0
0|0|3|0|3|0|0|0|0|0
0|0|0|3|0|3|0|0|0|0
0|0|3|0|0|0|3|0|0|0
0|0|0|0|0|3|0|3|0|0
0|0|0|3|0|3|3|0|0|0
0|0|3|3|3|0|0|0|0|0
0|0|0|3|0|0|0|0|0|0
0|0|0|0|0|0|0|0|0|0
0|0|0|0|0|0|0|0|0|0
```

Expected Output ASCII representation:

```
0|0|0|0|0|0|0|0|0|0
0|0|3|0|3|0|0|0|0|0
0|0|0|3|0|3|0|0|0|0
0|0|3|0|0|0|3|0|0|0
0|0|0|0|0|3|4|3|0|0
0|0|0|3|0|3|3|0|0|0
0|0|3|3|3|0|0|0|0|0
0|0|0|3|0|0|0|0|0|0
0|0|0|0|0|0|0|0|0|0
0|0|0|0|0|0|0|0|0|0
```

Your `transform` function was correct on this example! Think about why it worked on this and not others.

Example 2:

Input ASCII representation:

0|0|0|0|0|3|0|0|0|0
0|0|0|0|3|0|0|0|0|0
0|3|3|0|3|3|0|3|0|0
3|0|0|3|0|0|3|0|3|0
0|0|0|3|0|0|3|3|0|0
0|0|0|3|0|0|3|0|0|0
0|0|0|3|0|0|3|0|0|0
0|0|0|0|3|3|0|3|0|0
0|0|0|0|0|0|0|0|3|0
0|0|0|0|0|0|0|0|0|0

Expected Output ASCII representation:

0|0|0|0|0|3|0|0|0|0
0|0|0|0|3|0|0|0|0|0
0|3|3|0|3|3|0|3|0|0
3|0|0|3|4|4|3|4|3|0
0|0|0|3|4|4|3|3|0|0
0|0|0|3|4|4|3|0|0|0
0|0|0|3|4|4|3|0|0|0
0|0|0|0|3|3|0|3|0|0
0|0|0|0|0|0|0|0|3|0
0|0|0|0|0|0|0|0|0|0

Incorrect Transformed Output ASCII representation:

0|0|0|0|0|3|0|0|0|0
0|0|0|0|3|0|0|0|0|0
0|3|3|0|3|3|0|3|0|0
3|0|0|3|0|0|3|4|3|0
0|0|0|3|0|0|3|3|0|0
0|0|0|3|0|0|3|0|0|0
0|0|0|3|0|0|3|0|0|0
0|0|0|0|3|3|0|3|0|0

0|0|0|0|0|0|0|0|3|0
0|0|0|0|0|0|0|0|0|0

Color changes between the Expected Output ASCII and your Transformed Output:

--	--	--	--	--	--	--	--	--	--
--	--	--	--	--	--	--	--	--	--
--	--	--	--	4 -> 0	4 -> 0	--	--	--	--
--	--	--	--	4 -> 0	4 -> 0	--	--	--	--
--	--	--	--	4 -> 0	4 -> 0	--	--	--	--
--	--	--	--	4 -> 0	4 -> 0	--	--	--	--
--	--	--	--	--	--	--	--	--	--
--	--	--	--	--	--	--	--	--	--
--	--	--	--	--	--	--	--	--	--

Example 3:

Input ASCII representation:

0|0|0|0|0|0|0|0|0|0
0|0|3|3|3|3|0|0|0|0
0|0|3|0|0|3|0|0|0|0
0|0|3|0|0|3|0|3|0|0
0|0|3|3|3|3|3|3|0|0
0|0|0|3|0|0|0|0|3|0
0|0|0|3|0|0|0|3|3|0
0|0|0|3|3|0|0|3|0|3
0|0|0|3|0|3|0|0|3|0
0|0|0|0|3|0|0|0|0|0

Expected Output ASCII representation:

0|0|0|0|0|0|0|0|0|0
0|0|3|3|3|3|0|0|0|0
0|0|3|4|4|3|0|0|0|0
0|0|3|4|4|3|0|3|0|0
0|0|3|3|3|3|3|3|3|0
0|0|0|3|0|0|0|0|3|0
0|0|0|3|0|0|0|3|3|0
0|0|0|3|3|0|0|3|4|3
0|0|0|3|4|3|0|0|3|0
0|0|0|0|3|0|0|0|0|0

Incorrect Transformed Output ASCII representation:

0|0|0|0|0|0|0|0|0|0
0|0|3|3|3|3|0|0|0|0
0|0|3|0|0|3|0|0|0|0
0|0|3|0|0|3|0|3|0|0
0|0|3|3|3|3|3|3|3|0
0|0|0|3|0|0|0|0|3|0
0|0|0|3|0|0|0|3|3|0
0|0|0|3|3|0|0|3|4|3
0|0|0|3|4|3|0|0|3|0
0|0|0|0|3|0|0|0|0|0

Color changes between the Expected Output ASCII and your Transformed Output:

[illegible]

0|0|0|0|0|0|0|0|3|4|4|4|4|4|4|3|0|0|0|0
0|0|0|0|0|0|0|0|3|4|4|4|4|4|4|3|0|0|0|0
0|0|3|0|0|0|0|0|3|3|3|3|3|3|3|3|0|0|0|0
0|0|0|0|0|0|0|0|3|0|0|0|0|0|0|0|0|0|0|0
0|0|0|0|0|0|0|0|3|3|3|0|0|0|0|3|0|3|0|0
0|0|0|0|0|0|0|3|3|4|4|3|0|0|3|0|0|0|0|0|0
0|0|0|0|0|0|0|3|4|4|3|3|0|0|3|0|0|3|0|0
0|0|0|0|0|0|0|3|3|3|3|0|3|0|0|3|3|3|0|0
0|0|0|0|0|0|0|0|0|0|3|0|0|0|0|3|4|3|0|0
0|0|0|0|0|0|0|0|0|0|0|0|3|0|0|3|3|3|0|0
0|0|0|0|0|0|0|0|0|0|0|0|0|3|0|0|0|0|0|0
0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0
0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0

Incorrect Transformed Output ASCII representation:

0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0
0|0|0|0|0|0|0|0|3|0|0|0|0|0|0|0|0|0|0|0
0|0|0|0|3|3|3|3|4|3|3|0|0|0|0|0|0|0|0|0
0|0|0|0|0|0|0|0|3|4|3|0|0|0|0|0|0|0|3|0
0|0|0|0|0|0|0|0|3|3|3|3|3|3|3|3|0|0|0|0
0|0|0|0|0|0|0|0|3|0|0|0|0|0|0|0|3|0|0|0
0|0|0|0|3|0|0|0|3|0|0|0|0|0|0|3|0|0|0|0
0|0|0|0|0|0|0|0|3|0|0|0|0|0|0|3|0|0|0|0
0|0|0|0|0|0|0|0|3|0|0|0|0|0|0|3|0|0|0|0
0|0|3|0|0|0|0|0|3|3|3|3|3|3|3|3|0|0|0|0
0|0|0|0|0|0|0|0|3|0|0|0|0|0|0|0|0|0|0|0
0|0|0|0|0|0|0|0|3|3|3|0|0|0|0|3|0|3|0|0
0|0|0|0|0|0|3|3|0|0|3|0|0|3|0|0|0|0|0|0
0|0|0|0|0|0|0|3|0|0|3|3|0|0|3|0|0|3|0|0
0|0|0|0|0|0|0|3|3|3|3|0|3|0|0|3|3|3|0|0
0|0|0|0|0|0|0|0|0|0|3|0|0|0|0|3|4|3|0|0
0|0|0|0|0|0|0|0|0|0|0|0|3|0|0|3|3|3|0|0
0|0|0|0|0|0|0|0|0|0|0|0|0|3|0|0|0|0|0|0
0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0
0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0

Color changes between the Expected Output ASCII and your

[illegible]

								0	0										
								4	4										
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	>	>	-	-	-	-	-	-	-	-	-	-
								0	0										
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

Your previous incorrect response #1:

Let me analyze the patterns in the examples:

Example 1:

- Input has a pattern of green (3) cells in a diagonal shape
- In the output, two black (0) cells inside the pattern are changed to yellow (4)
- These yellow cells form a 2x1 group
- The yellow cells appear where there are empty spaces surrounded by green cells

Example 2:

- Similar diagonal-like pattern of green cells
- One cell is changed to yellow, and it appears in an empty space surrounded by green cells
- The yellow cell also forms a pattern with nearby green cells

Example 3:

- Has a more complex structure of green cells
- In the output, multiple cells are changed from black to yellow

- The yellow cells appear in rectangular regions that are surrounded by green cells
- Forms a 2x3 block of yellow cells in the middle
- Also has a 2x2 block of yellow cells below
- All yellow cells are surrounded by green cells

Example 4:

- Has patterns of green cells forming various shapes
- In the output, empty spaces within these shapes are filled with yellow cells
- The yellow cells form 2x1 and 2x2 patterns
- Again, all yellow cells are surrounded by green cells

Example 5:

- Shows a larger pattern with multiple structures
- Yellow cells fill in empty spaces that are surrounded by green cells
- Forms multiple rectangular regions (6x6, 2x2, etc.)

Common patterns:

1. The transformation only changes black (0) cells to yellow (4)
2. Yellow cells only appear in spaces that are surrounded by green (3) cells
3. Yellow cells form rectangular regions (1x2, 2x2, 2x3, 6x6, etc.)
4. All yellow cells must be part of a rectangular group
5. The rectangular groups must be completely surrounded by green cells

The rule appears to be:

1. Find all black cells that are surrounded by green cells
2. If these black cells can form a rectangular group (at least 2 cells)
3. And if this rectangular group is completely surrounded by green cells
4. Then fill all cells in that rectangular group with yellow (4)

Let me implement this transformation.

```
import numpy as np

def is_surrounded_by_green(grid, i, j):
    \"\"\"Check if cell is surrounded by green (3) cells\"\"\"
    rows, cols = grid.shape
    neighbors = []
    for di in [-1, 0, 1]:
        for dj in [-1, 0, 1]:
            if di == 0 and dj == 0:
```

```

        continue
    ni, nj = i + di, j + dj
    if 0 <= ni < rows and 0 <= nj < cols:
        neighbors.append(grid[ni, nj])
return all(n == 3 for n in neighbors if n != 0)

def find_rectangle(grid, start_i, start_j):
    """Find the largest rectangle of black cells starting at (i,j)"""
    rows, cols = grid.shape
    if grid[start_i, start_j] != 0:
        return None

    # Find max width
    width = 0
    j = start_j
    while j < cols and grid[start_i, j] == 0:
        width += 1
        j += 1

    # Find max height
    height = 0
    i = start_i
    while i < rows:
        # Check if current row has enough black cells
        if all(grid[i, start_j + dj] == 0 for dj in range(width)):
            height += 1
            i += 1
        else:
            break

    if width < 2 and height < 2: # Must be at least 2 cells
        return None

    # Check if rectangle is surrounded by green
    for i in range(start_i - 1, start_i + height + 1):
        for j in range(start_j - 1, start_j + width + 1):
            if i < 0 or i >= rows or j < 0 or j >= cols:
                continue
            if (i == start_i - 1 or i == start_i + height or
                j == start_j - 1 or j == start_j + width):
                if grid[i, j] != 3:
                    return None

    return (start_i, start_j, height, width)

def transform(grid_lst: list[list[int]]) -> list[list[int]]:
    grid = np.array(grid_lst)
    rows, cols = grid.shape

    # Find all possible rectangles
    rectangles = []
    for i in range(rows):
        for j in range(cols):

```

```

        rect = find_rectangle(grid, i, j)
        if rect:
            rectangles.append(rect)

# Fill rectangles with yellow (4)
for i, j, height, width in rectangles:
    grid[i:i+height, j:j+width] = 4

return grid.tolist()

```

The results of using your `transform` function on each example:

Example 0:

Input ASCII representation:

```

0|0|0|0|0|0
0|0|3|0|0|0
0|3|0|3|0|0
0|0|3|0|3|0
0|0|0|3|0|0
0|0|0|0|0|0

```

Expected Output ASCII representation:

```

0|0|0|0|0|0
0|0|3|0|0|0
0|3|4|3|0|0
0|0|3|4|3|0
0|0|0|3|0|0
0|0|0|0|0|0

```

Incorrect Transformed Output ASCII representation:

```

0|0|0|0|0|0
0|0|3|0|0|0

```

0|3|0|3|0|0
0|0|3|0|3|0
0|0|0|3|0|0
0|0|0|0|0|0

Color changes between the Expected Output ASCII and your Transformed Output:

--	--	--	--	--	--
--	--	4 -> 0	--	--	--
--	--	--	4 -> 0	--	--
--	--	--	--	--	--
--	--	--	--	--	--

Example 1:

Input ASCII representation:

0|0|0|0|0|0|0|0|0|0
0|0|3|0|3|0|0|0|0|0
0|0|0|3|0|3|0|0|0|0
0|0|3|0|0|0|3|0|0|0
0|0|0|0|0|3|0|3|0|0
0|0|0|3|0|3|3|0|0|0
0|0|3|3|3|0|0|0|0|0
0|0|0|3|0|0|0|0|0|0
0|0|0|0|0|0|0|0|0|0
0|0|0|0|0|0|0|0|0|0

Expected Output ASCII representation:

0|0|0|0|0|0|0|0|0|0
0|0|3|0|3|0|0|0|0|0
0|0|0|3|0|3|0|0|0|0

Example 2:

Input ASCII representation:

```
0|0|0|0|0|3|0|0|0|0
0|0|0|0|3|0|0|0|0|0
0|3|3|0|3|3|0|3|0|0
3|0|0|3|0|0|3|0|3|0
0|0|0|3|0|0|3|3|0|0
0|0|0|3|0|0|3|0|0|0
0|0|0|3|0|0|3|0|0|0
0|0|0|0|3|3|0|3|0|0
0|0|0|0|0|0|0|0|3|0
0|0|0|0|0|0|0|0|0|0
```

Expected Output ASCII representation:

```
0|0|0|0|0|3|0|0|0|0
0|0|0|0|3|0|0|0|0|0
0|3|3|0|3|3|0|3|0|0
3|0|0|3|4|4|3|4|3|0
0|0|0|3|4|4|3|3|0|0
0|0|0|3|4|4|3|0|0|0
0|0|0|3|4|4|3|0|0|0
0|0|0|0|3|3|0|3|0|0
0|0|0|0|0|0|0|0|3|0
0|0|0|0|0|0|0|0|0|0
```

Incorrect Transformed Output ASCII representation:

```
0|0|0|0|0|3|0|0|0|0
0|0|0|0|3|0|0|0|0|0
0|3|3|0|3|3|0|3|0|0
3|0|0|3|0|0|3|0|3|0
0|0|0|3|0|0|3|3|0|0
```

0|0|0|3|0|0|3|0|0|0
0|0|0|3|0|0|3|0|0|0
0|0|0|0|3|3|0|3|0|0
0|0|0|0|0|0|0|0|3|0
0|0|0|0|0|0|0|0|0|0

Color changes between the Expected Output ASCII and your Transformed Output:

--	--	--	--	--	--	--	--	--	--
--	--	--	--	--	--	--	--	--	--
--	--	--	--	4 -> 0	4 -> 0	--	4 -> 0	--	--
--	--	--	--	4 -> 0	4 -> 0	--	--	--	--
--	--	--	--	4 -> 0	4 -> 0	--	--	--	--
--	--	--	--	4 -> 0	4 -> 0	--	--	--	--
--	--	--	--	--	--	--	--	--	--
--	--	--	--	--	--	--	--	--	--
--	--	--	--	--	--	--	--	--	--

Example 3:

Input ASCII representation:

0|0|0|0|0|0|0|0|0|0
0|0|3|3|3|3|0|0|0|0
0|0|3|0|0|3|0|0|0|0
0|0|3|0|0|3|0|3|0|0
0|0|3|3|3|3|3|3|3|0
0|0|0|3|0|0|0|0|3|0
0|0|0|3|0|0|0|3|3|0
0|0|0|3|3|0|0|3|0|3
0|0|0|3|0|3|0|0|3|0
0|0|0|0|3|0|0|0|0|0

Expected Output ASCII representation:

0|0|0|0|0|0|0|0|0|0
0|0|3|3|3|3|0|0|0|0
0|0|3|4|4|3|0|0|0|0
0|0|3|4|4|3|0|3|0|0
0|0|3|3|3|3|3|3|3|0
0|0|0|3|0|0|0|0|3|0
0|0|0|3|0|0|0|3|3|0
0|0|0|3|3|0|0|3|4|3
0|0|0|3|4|3|0|0|3|0
0|0|0|0|3|0|0|0|0|0

Incorrect Transformed Output ASCII representation:

0|0|0|0|0|0|0|0|0|0
0|0|3|3|3|3|0|0|0|0
0|0|3|4|4|3|0|0|0|0
0|0|3|4|4|3|0|3|0|0
0|0|3|3|3|3|3|3|3|0
0|0|0|3|0|0|0|0|3|0
0|0|0|3|0|0|0|3|3|0
0|0|0|3|3|0|0|3|0|3
0|0|0|3|0|3|0|0|3|0
0|0|0|0|3|0|0|0|0|0

Color changes between the Expected Output ASCII and your Transformed Output:

[illegible]

0|0|0|0|0|0|0|0|3|3|3|3|3|3|3|0|0|0|0|0
0|0|0|0|0|0|0|0|3|4|4|4|4|4|4|3|0|0|0|0|0
0|0|0|0|3|0|0|0|3|4|4|4|4|4|4|3|0|0|0|0|0
0|0|0|0|0|0|0|0|3|4|4|4|4|4|4|3|0|0|0|0|0
0|0|0|0|0|0|0|0|3|4|4|4|4|4|4|3|0|0|0|0|0
0|0|3|0|0|0|0|0|3|3|3|3|3|3|3|3|0|0|0|0|0
0|0|0|0|0|0|0|0|3|0|0|0|0|0|0|0|0|0|0|0|0
0|0|0|0|0|0|0|0|3|3|3|0|0|0|0|3|0|3|0|0|0
0|0|0|0|0|0|3|3|4|4|3|0|0|3|0|0|0|0|0|0|0
0|0|0|0|0|0|0|3|4|4|3|3|0|0|3|0|0|3|0|0|0
0|0|0|0|0|0|0|3|3|3|3|0|3|0|0|3|3|3|0|0|0
0|0|0|0|0|0|0|0|0|0|3|0|0|0|0|3|4|3|0|0|0
0|0|0|0|0|0|0|0|0|0|0|0|0|3|0|0|3|3|3|0|0|0
0|0|0|0|0|0|0|0|0|0|0|0|0|3|0|0|0|0|0|0|0
0|0
0|0

Incorrect Transformed Output ASCII representation:

0|0
0|0|0|0|0|0|0|0|3|0|0|0|0|0|0|0|0|0|0|0|0
0|0|0|0|3|3|3|3|0|3|3|0|0|0|0|0|0|0|0|0|0
0|0|0|0|0|0|0|0|3|0|3|0|0|0|0|0|0|0|0|3|0
0|0|0|0|0|0|0|0|3|3|3|3|3|3|3|3|3|0|0|0|0
0|0|0|0|0|0|0|0|3|4|4|4|4|4|4|3|0|0|0|0|0
0|0|0|0|3|0|0|0|3|4|4|4|4|4|4|3|0|0|0|0|0
0|0|0|0|0|0|0|0|3|4|4|4|4|4|4|3|0|0|0|0|0
0|0|0|0|0|0|0|0|3|4|4|4|4|4|4|3|0|0|0|0|0
0|0|3|0|0|0|0|0|3|3|3|3|3|3|3|3|3|0|0|0|0
0|0|0|0|0|0|0|0|3|0|0|0|0|0|0|0|0|0|0|0|0
0|0|0|0|0|0|0|0|3|3|3|0|0|0|0|3|0|3|0|0|0
0|0|0|0|0|0|3|3|0|0|3|0|0|3|0|0|0|0|0|0|0
0|0|0|0|0|0|0|3|0|0|3|3|0|0|3|0|0|3|0|0|0
0|0|0|0|0|0|0|3|3|3|3|0|3|0|0|3|3|3|0|0|0
0|0|0|0|0|0|0|0|0|0|3|0|0|0|0|3|0|3|0|0|0
0|0|0|0|0|0|0|0|0|0|0|0|0|3|0|0|3|3|3|0|0|0
0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|3|0|0|0|0|0|0
0|0
0|0

[illegible]

-	-	-	-	-	-	-	-	--	--	-	-	-	-	-	-	--	-	-	-
-	-	-	-	-	-	-	-			-	-	-	-	-	-		-	-	-
-	-	-	-	-	-	-	-	--	--	-	-	-	-	-	-	4	-	-	-
-	-	-	-	-	-	-	-			-	-	-	-	-	-	>	-	-	-
-	-	-	-	-	-	-	-			-	-	-	-	-	-	0	-	-	-
-	-	-	-	-	-	-	-	--	--	-	-	-	-	-	-	--	-	-	-
-	-	-	-	-	-	-	-			-	-	-	-	-	-		-	-	-
-	-	-	-	-	-	-	-	--	--	-	-	-	-	-	-	--	-	-	-
-	-	-	-	-	-	-	-			-	-	-	-	-	-		-	-	-
-	-	-	-	-	-	-	-	--	--	-	-	-	-	-	-	--	-	-	-
-	-	-	-	-	-	-	-			-	-	-	-	-	-		-	-	-
-	-	-	-	-	-	-	-	--	--	-	-	-	-	-	-	--	-	-	-
-	-	-	-	-	-	-	-			-	-	-	-	-	-		-	-	-

Recall that you should start by reasoning to determine what the issue is in `<fix_reasoning>` `</fix_reasoning>` tags. Also recall that the problem could be a bug in the code and/or an issue with your previous understanding of the transformation rule.

If you notice an issue with your previous understanding of the transformation rule, you'll need to do further analysis (including analyzing properties of the example inputs and outputs) to determine exactly what the correct transformation rule is.

Once you are done reasoning, rewrite the code to fix the issue. Return the code in triple backticks (`python` and `then`).

If your attempted fix fails, you'll be called again (in the same way) to continue debugging. So, if print statements would help you debug, you can include them in your code.